

SLOT 13: OAuth and User Authentication

Tóm tắt

- **OAuth** là một tiêu chuẩn mở giúp xác thực người dùng mà không cần lưu trữ mật khẩu, thường sử dụng Google, Facebook, GitHub, v.v.
- **Passport.js** là middleware xác thực cho Node.js, hỗ trợ nhiều phương thức đăng nhập, bao gồm OAuth.

CÁC NỘI DUNG TRỌNG TÂM

1. OAuth và Xác Thực Người Dùng

- **OAuth là gì?:** Giao thức cấp quyền truy cập an toàn mà không cần chia sẻ mật khẩu.
- **Cách hoạt động:** Người dùng đăng nhập qua OAuth Provider (Google, Facebook, GitHub...), server nhận access token để xác thực.
- **Lợi ích:** Bảo mật cao hơn, tiện lợi, không cần lưu mật khẩu.

2. Setup Server với Passport OAuth Modules

1. **Cài đặt môi trường:**
 - Node.js, Express, MongoDB, Passport.js, dotenv.
2. **Cấu hình Express Server:**
 - Dùng `express-session` để lưu session.
 - Kết nối MongoDB bằng `mongoose`.
3. **Thiết lập Passport OAuth (Google, Facebook, GitHub...)**
 - Dùng `passport-google-oauth20` hoặc các strategy khác.
 - Định nghĩa `passport.js` (`GoogleStrategy`, `serialize & deserialize user`).
4. **Tạo routes xác thực**
 - `/auth/google`: Chuyển hướng đến Google.
 - `/auth/google/callback`: Nhận dữ liệu user.
 - `/logout`: Xóa session đăng nhập.
5. **Quản lý user với MongoDB**
 - Lưu thông tin user vào database.
 - Kiểm tra xem user đã tồn tại hay chưa.
6. **Tạo giao diện đơn giản với EJS**
 - Trang đăng nhập.
 - Dashboard sau khi đăng nhập thành công.
7. **Kiểm thử & xử lý lỗi**
 - Dùng Postman hoặc trình duyệt.
 - Debug lỗi OAuth: `redirect URI mismatch`, `unauthorized client`.

ỨNG DỤNG TRONG THỰC TIỄN

OAuth giúp ứng dụng xác thực người dùng an toàn mà không yêu cầu họ nhập mật khẩu trực tiếp vào hệ thống của bạn. Điều này rất quan trọng khi phát triển các ứng dụng Node.js có liên quan đến quản lý tài khoản người dùng.

- ◆ OAuth giúp đăng nhập an toàn và không cần lưu mật khẩu.
- ◆ Passport.js giúp tích hợp OAuth dễ dàng với nhiều nhà cung cấp như Google, Facebook.
- ◆ Ứng dụng có thể mở rộng để sử dụng JWT, API bảo mật và quản lý người dùng.

1. Ứng dụng thực tế của OAuth trong Node.js

1.1. Xác thực người dùng trong Web App

- Sử dụng OAuth để cho phép người dùng đăng nhập bằng Google, Facebook, GitHub thay vì đăng ký tài khoản mới.
- Giảm thiểu việc lưu trữ mật khẩu của người dùng, tăng cường bảo mật.

Ví dụ thực tế:

- **Trello, Asana:** Cho phép đăng nhập bằng Google, Microsoft.
- **Medium, Dev.to:** Cho phép đăng nhập bằng GitHub, Google.

1.2. Tích hợp OAuth vào ứng dụng di động

- Ứng dụng di động có thể sử dụng OAuth để xác thực với Google hoặc Apple ID.
- Khi người dùng mở app, họ có thể chọn "**Đăng nhập bằng Google**", giúp trải nghiệm nhanh hơn.

Ví dụ thực tế:

- **Spotify, Netflix:** Sử dụng OAuth để liên kết tài khoản Google hoặc Facebook.

1.3. API Authentication (Xác thực API)

- Nếu bạn xây dựng API RESTful, OAuth giúp xác thực API thay vì sử dụng session hoặc JWT thông thường.
- Người dùng đăng nhập và hệ thống cấp **Access Token**, token này có thể dùng để gọi API mà không cần nhập mật khẩu.

Ví dụ thực tế:

- **GitHub API, Google API:** Yêu cầu xác thực bằng OAuth khi sử dụng API của họ.

1.4. Single Sign-On (SSO)

- Một công ty có nhiều ứng dụng khác nhau, thay vì yêu cầu mỗi ứng dụng có một hệ thống đăng nhập riêng, có thể dùng OAuth để triển khai **Single Sign-On (SSO)**.
- SSO giúp người dùng đăng nhập một lần nhưng có thể truy cập vào nhiều ứng dụng.

Ví dụ thực tế:

- **Google Account SSO:** Khi bạn đăng nhập vào Gmail, bạn cũng có thể truy cập Google Drive, YouTube mà không cần nhập lại mật khẩu.

2. Xây dựng ứng dụng Node.js với OAuth và Passport.js

Bây giờ chúng ta sẽ phát triển một ứng dụng thực tế sử dụng OAuth để đăng nhập bằng Google.

2.1. Chức năng chính của ứng dụng

- ✓ Đăng nhập bằng Google OAuth
- ✓ Lưu thông tin người dùng vào MongoDB
- ✓ Quản lý phiên đăng nhập với Session
- ✓ API riêng yêu cầu xác thực bằng Access Token

2.2. Công nghệ sử dụng

- **Node.js + Express.js:** Framework backend
- **MongoDB + Mongoose:** Lưu thông tin người dùng
- **Passport.js:** Middleware xác thực OAuth2
- **Session + Cookie:** Quản lý phiên đăng nhập

2.3. Cài đặt môi trường

```
mkdir node-oauth-app
cd node-oauth-app
npm init -y
npm install express mongoose passport passport-google-oauth20 express-session
dotenv
```

2.4. Cấu hình Google OAuth

1. Truy cập Google Developer Console
2. Tạo ứng dụng OAuth và lấy **Client ID + Client Secret**
3. Cấu hình **Redirect URI**:

```
http://localhost:3000/auth/google/callback
```

4. Tạo file .env để lưu thông tin OAuth:

```
env

GOOGLE_CLIENT_ID=your_client_id
GOOGLE_CLIENT_SECRET=your_client_secret
CALLBACK_URL=http://localhost:3000/auth/google/callback
MONGO_URI=mongodb://localhost:27017/oauth-users
SESSION_SECRET=your_secret
```

2.5. Kết nối MongoDB

Tạo file db.js để kết nối MongoDB:

```
javascript

const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI, { useNewUrlParser:
true, useUnifiedTopology: true });
    console.log("MongoDB Connected...");
  } catch (err) {
    console.error(err);
  }
};

module.exports = connectDB;
```

2.6. Xây dựng mô hình User

Tạo file models/User.js để lưu thông tin người dùng:

```
javascript

const mongoose = require("mongoose");

const UserSchema = new mongoose.Schema({
  googleId: String,
  displayName: String,
  email: String,
  photo: String
});

module.exports = mongoose.model("User", UserSchema);
```

2.7. Cấu hình Passport OAuth

Tạo file passport.js:

```
javascript
```

```

const passport = require("passport");
const GoogleStrategy = require("passport-google-oauth20").Strategy;
const User = require("../models/User");

passport.use(new GoogleStrategy({
  clientID: process.env.GOOGLE_CLIENT_ID,
  clientSecret: process.env.GOOGLE_CLIENT_SECRET,
  callbackURL: process.env.CALLBACK_URL
}, async (accessToken, refreshToken, profile, done) => {
  let user = await User.findOne({ googleId: profile.id });

  if (!user) {
    user = await User.create({
      googleId: profile.id,
      displayName: profile.displayName,
      email: profile.emails[0].value,
      photo: profile.photos[0].value
    });
  }

  return done(null, user);
}));

passport.serializeUser((user, done) => done(null, user.id));
passport.deserializeUser(async (id, done) => {
  const user = await User.findById(id);
  done(null, user);
});

```

2.8. Xây dựng Server với Express

Tạo server.js:

```

javascript

require("dotenv").config();
const express = require("express");
const mongoose = require("mongoose");
const session = require("express-session");
const passport = require("passport");
require("../db")();
require("../passport");

const app = express();

app.use(session({
  secret: process.env.SESSION_SECRET,
  resave: false,
  saveUninitialized: true
}));

app.use(passport.initialize());
app.use(passport.session());

// Route chính
app.get("/", (req, res) => {
  res.send('<a href="/auth/google">Đăng nhập với Google</a>');
});

// Xác thực OAuth

```

```

app.get("/auth/google",
  passport.authenticate("google", { scope: ["profile", "email"] })
);

app.get("/auth/google/callback",
  passport.authenticate("google", { failureRedirect: "/" }),
  (req, res) => {
    res.redirect("/dashboard");
  }
);

// Dashboard
app.get("/dashboard", (req, res) => {
  if (!req.isAuthenticated()) {
    return res.redirect("/");
  }
  res.send(`<h1>Chào ${req.user.displayName}</h1> <a href="/logout">Đăng
xuất</a>`);
});

// Đăng xuất
app.get("/logout", (req, res) => {
  req.logout(() => {
    res.redirect("/");
  });
});

// Chạy server
app.listen(3000, () => console.log("Server chạy trên
http://localhost:3000"));

```

2.9. Chạy ứng dụng

```
node server.js
```

Truy cập <http://localhost:3000> và thử đăng nhập bằng Google.

.

MỘT SỐ NỘI DUNG CHI TIẾT

OAuth (Open Authorization) là một tiêu chuẩn mở giúp các ứng dụng bên thứ ba có thể truy cập dữ liệu của người dùng từ một dịch vụ mà không cần biết mật khẩu của họ.

1. Hiểu về OAuth

1.1. OAuth là gì?

OAuth (Open Authorization) là một giao thức cấp quyền cho phép các ứng dụng bên thứ ba truy cập vào tài nguyên của người dùng trên một dịch vụ khác mà không cần tiết lộ thông tin đăng nhập của họ.

Ví dụ, bạn có thể sử dụng tài khoản Google để đăng nhập vào một website thay vì tạo một tài khoản mới. Website đó sử dụng OAuth để xác thực danh tính của bạn.

1.2. OAuth 2.0 hoạt động như thế nào?

OAuth 2.0 sử dụng **Access Token** để cấp quyền truy cập. Dưới đây là các bước xác thực cơ bản:

1. **Người dùng chọn phương thức đăng nhập** (ví dụ: “Đăng nhập với Google”).
2. **Ứng dụng web chuyển hướng người dùng** đến trang đăng nhập của OAuth provider (Google, Facebook, GitHub, v.v.).
3. **Người dùng nhập thông tin và đồng ý cấp quyền.**
4. **OAuth provider xác thực người dùng** và cung cấp một **Authorization Code** cho ứng dụng.
5. **Ứng dụng đổi Authorization Code lấy Access Token** từ OAuth provider.
6. **Ứng dụng sử dụng Access Token** để truy xuất thông tin người dùng từ OAuth provider.
7. **Ứng dụng xác thực người dùng và tạo phiên làm việc.**

1.3. Các thành phần chính của OAuth 2.0

- **Resource Owner:** Người dùng đang đăng nhập.
- **Client (Ứng dụng):** Ứng dụng bên thứ ba yêu cầu quyền truy cập.
- **Authorization Server:** Máy chủ xác thực và cấp Access Token.
- **Resource Server:** API cung cấp dữ liệu của người dùng (ví dụ: API của Google, Facebook).

2. Thiết lập Server sử dụng Passport OAuth Modules

Passport.js là một middleware xác thực phổ biến trong Node.js, hỗ trợ nhiều phương thức xác thực, bao gồm **OAuth 2.0**.

2.1. Cài đặt môi trường

Trước tiên, bạn cần tạo một dự án Node.js và cài đặt các package cần thiết.

```
mkdir passport-oauth-example
cd passport-oauth-example
npm init -y
npm install express passport passport-google-oauth20 express-session dotenv
```

- `express`: Framework web cho Node.js.
- `passport`: Middleware xác thực.
- `passport-google-oauth20`: Module Passport để xác thực OAuth2 với Google.
- `express-session`: Quản lý session.
- `dotenv`: Quản lý biến môi trường.

2.2. Cấu hình OAuth Provider

Để sử dụng Google OAuth, bạn cần đăng ký ứng dụng tại Google Developer Console.

Bước 1: Tạo Credentials

- Truy cập **Google Cloud Console** → **APIs & Services** → **Credentials**.
- Chọn **Create Credentials** → **OAuth 2.0 Client ID**.
- Chọn **Application type: Web application**.
- Thêm **Authorized redirect URIs**:

```
http://localhost:3000/auth/google/callback
```

- Sao chép **Client ID** và **Client Secret**.

Bước 2: Lưu thông tin OAuth vào `.env`

Tạo file `.env` và thêm thông tin:

```
env
```

```
GOOGLE_CLIENT_ID=your_google_client_id
GOOGLE_CLIENT_SECRET=your_google_client_secret
CALLBACK_URL=http://localhost:3000/auth/google/callback
SESSION_SECRET=your_secret_key
```

2.3. Cấu hình Passport OAuth

Tạo tệp `server.js` và cấu hình Passport:

```
javascript
```

```
require("dotenv").config();
const express = require("express");
const passport = require("passport");
```



```

const session = require("express-session");
const GoogleStrategy = require("passport-google-oauth20").Strategy;

const app = express();

// Cấu hình session
app.use(session({
  secret: process.env.SESSION_SECRET,
  resave: false,
  saveUninitialized: true
}));
app.use(passport.initialize());
app.use(passport.session());

// Cấu hình Passport với Google OAuth
passport.use(new GoogleStrategy({
  clientID: process.env.GOOGLE_CLIENT_ID,
  clientSecret: process.env.GOOGLE_CLIENT_SECRET,
  callbackURL: process.env.CALLBACK_URL
}, (accessToken, refreshToken, profile, done) => {
  return done(null, profile);
}));

// Serialize & Deserialize User
passport.serializeUser((user, done) => done(null, user));
passport.deserializeUser((obj, done) => done(null, obj));

// Routes
app.get("/", (req, res) => {
  res.send('<a href="/auth/google">Đăng nhập với Google</a>');
});

// Route để bắt đầu xác thực OAuth
app.get("/auth/google",
  passport.authenticate("google", { scope: ["profile", "email"] })
);

// Route callback từ Google
app.get("/auth/google/callback",
  passport.authenticate("google", { failureRedirect: "/" }),
  (req, res) => {
    res.redirect("/dashboard");
  }
);

// Route dashboard hiển thị thông tin người dùng
app.get("/dashboard", (req, res) => {
  if (!req.isAuthenticated()) {
    return res.redirect("/");
  }
  res.send(`<h1>Chào ${req.user.displayName}</h1> <a href="/logout">Đăng
xuất</a>`);
});

// Route đăng xuất
app.get("/logout", (req, res) => {
  req.logout() => {
    res.redirect("/");
  }
});

// Chạy server
app.listen(3000, () => {

```

```
    console.log("Server đang chạy trên http://localhost:3000");  
  });
```

2.4. Chạy ứng dụng

Sau khi cấu hình xong, chạy server:

```
node server.js
```

Mở trình duyệt và truy cập `http://localhost:3000`, sau đó thử đăng nhập với Google.

3. Giải thích mã nguồn

- **Cấu hình Passport:**
 - `passport.use(new GoogleStrategy(...))`: Định nghĩa chiến lược xác thực.
 - `callbackURL`: Địa chỉ callback sau khi xác thực thành công.
- **Session Handling:**
 - `passport.serializeUser()` và `passport.deserializeUser()` dùng để lưu và lấy thông tin người dùng từ session.
- **Các Routes chính:**
 - `/auth/google`: Chuyển hướng người dùng đến Google để xác thực.
 - `/auth/google/callback`: Xử lý phản hồi từ Google.
 - `/dashboard`: Hiển thị thông tin người dùng sau khi đăng nhập.
 - `/logout`: Đăng xuất và xóa session.

4. Mở rộng ứng dụng

Bạn có thể mở rộng ứng dụng với các OAuth providers khác như:

- **Facebook**: Dùng `passport-facebook`.
- **GitHub**: Dùng `passport-github2`.

Ví dụ, cài đặt thêm Facebook OAuth:

```
npm install passport-facebook
```

Sau đó, cấu hình `passport.use(new FacebookStrategy(...))` tương tự như Google.

5. Tổng kết

- **OAuth** giúp xác thực an toàn, tránh việc lưu trữ mật khẩu của người dùng.
- **Passport.js** hỗ trợ nhiều **OAuth providers** như Google, Facebook, GitHub.
- Ứng dụng sử dụng **Access Token** để lấy thông tin người dùng từ nhà cung cấp OAuth.
- Cấu hình server sử dụng **Express, Passport và Session** để quản lý đăng nhập

Setup Server Using Passport OAuth Modules

Tóm tắt các bước chính:

1. **Cài đặt thư viện** (express, mongoose, passport, passport-google-oauth20, express-session, dotenv)
2. **Cấu hình biến môi trường** (.env)
3. **Kết nối MongoDB** (db.js)
4. **Tạo Model User** (models/User.js)
5. **Cấu hình Passport Google OAuth** (passport.js)
6. **Thiết lập server với Express** (server.js)
7. **Chạy server và kiểm thử đăng nhập bằng Google**

1. Tổng quan về OAuth và Passport.js

1.1. OAuth là gì?

OAuth (Open Authorization) là một giao thức ủy quyền mở cho phép ứng dụng cấp quyền truy cập vào dữ liệu của người dùng mà không cần chia sẻ mật khẩu. OAuth được sử dụng phổ biến để đăng nhập bằng Google, Facebook, GitHub, Twitter, v.v.

1.2. Passport.js là gì?

Passport.js là một middleware dành cho Node.js giúp xác thực người dùng thông qua nhiều phương thức khác nhau, bao gồm OAuth 2.0, Local Authentication, JWT, và nhiều phương thức khác.

2. Quy trình xác thực OAuth với Passport.js

Khi triển khai xác thực OAuth trên server Node.js với Passport.js, quá trình diễn ra như sau:

1. **Người dùng nhấn vào "Đăng nhập bằng Google/Facebook..."**
→ Ứng dụng chuyển hướng đến OAuth Provider.
2. **Người dùng cấp quyền truy cập ứng dụng**
→ OAuth Provider xác thực và cấp Authorization Code.
3. **Server gửi Authorization Code đến OAuth Provider để đổi Access Token**
→ OAuth Provider cấp Access Token (và có thể là Refresh Token).
4. **Server sử dụng Access Token để lấy thông tin người dùng**
→ Nếu lần đầu đăng nhập, hệ thống sẽ tạo user mới trong database.
5. **Lưu thông tin phiên đăng nhập bằng session hoặc JWT**
→ Người dùng được xác thực và có thể truy cập ứng dụng.

3. Thiết lập OAuth trong Node.js với Passport.js

Bây giờ chúng ta sẽ thiết lập server để xác thực bằng Google OAuth.

3.1. Cài đặt thư viện cần thiết

```
mkdir passport-oauth-server
cd passport-oauth-server
npm init -y
npm install express mongoose passport passport-google-oauth20 express-session
dotenv
```

3.2. Cấu hình môi trường

Tạo file `.env` để lưu thông tin OAuth Client:

```
env

GOOGLE_CLIENT_ID=your_google_client_id
GOOGLE_CLIENT_SECRET=your_google_client_secret
CALLBACK_URL=http://localhost:3000/auth/google/callback
MONGO_URI=mongodb://localhost:27017/passport-users
SESSION_SECRET=your_session_secret
```

4. Kết nối MongoDB

Tạo file `db.js` để kết nối MongoDB:

```
javascript

const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI, {
      useNewUrlParser: true,
      useUnifiedTopology: true
    });
    console.log("MongoDB Connected...");
  } catch (err) {
    console.error(err);
  }
};

module.exports = connectDB;
```

5. Xây dựng Model User

Tạo file `models/User.js` để lưu thông tin người dùng:

```
javascript

const mongoose = require("mongoose");

const UserSchema = new mongoose.Schema({
  googleId: String,
  displayName: String,
  email: String,
  photo: String
});
```

```
module.exports = mongoose.model("User", UserSchema);
```

6. Cấu hình Passport với Google OAuth

Tạo file `passport.js` để cấu hình Passport.js:

```
javascript

const passport = require("passport");
const GoogleStrategy = require("passport-google-oauth20").Strategy;
const User = require("../models/User");

passport.use(new GoogleStrategy({
  clientID: process.env.GOOGLE_CLIENT_ID,
  clientSecret: process.env.GOOGLE_CLIENT_SECRET,
  callbackURL: process.env.CALLBACK_URL
}, async (accessToken, refreshToken, profile, done) => {
  let user = await User.findOne({ googleId: profile.id });

  if (!user) {
    user = await User.create({
      googleId: profile.id,
      displayName: profile.displayName,
      email: profile.emails[0].value,
      photo: profile.photos[0].value
    });
  }

  return done(null, user);
}));

passport.serializeUser((user, done) => {
  done(null, user.id);
});

passport.deserializeUser(async (id, done) => {
  const user = await User.findById(id);
  done(null, user);
});
```

7. Thiết lập Server Express

Tạo file `server.js`:

```
javascript

require("dotenv").config();
const express = require("express");
const mongoose = require("mongoose");
const session = require("express-session");
const passport = require("passport");
require("../db")();
require("../passport");

const app = express();

app.use(session({
  secret: process.env.SESSION_SECRET,
  resave: false,
```

```

        saveUninitialized: true
    }));

app.use(passport.initialize());
app.use(passport.session());

// Route chính
app.get("/", (req, res) => {
    res.send('<a href="/auth/google">Đăng nhập với Google</a>');
});

// Xác thực OAuth
app.get("/auth/google",
    passport.authenticate("google", { scope: ["profile", "email"] })
);

app.get("/auth/google/callback",
    passport.authenticate("google", { failureRedirect: "/" }),
    (req, res) => {
        res.redirect("/dashboard");
    }
);

// Dashboard
app.get("/dashboard", (req, res) => {
    if (!req.isAuthenticated()) {
        return res.redirect("/");
    }
    res.send(`<h1>Chào ${req.user.displayName}</h1> <a href="/logout">Đăng
xuất</a>`);
});

// Đăng xuất
app.get("/logout", (req, res) => {
    req.logout(() => {
        res.redirect("/");
    });
});

// Chạy server
app.listen(3000, () => console.log("Server chạy trên
http://localhost:3000"));

```

8. Chạy ứng dụng

node server.js

Truy cập <http://localhost:3000> và thử đăng nhập bằng Google.

9. Mở rộng OAuth với JWT

Nếu bạn không muốn sử dụng session, có thể sử dụng **JWT** để thay thế.

Cài đặt **jsonwebtoken**:

```
npm install jsonwebtoken
```

Cập nhật passport.js để trả về JWT Token:

```
javascript
```

```
const jwt = require("jsonwebtoken");

passport.use(new GoogleStrategy({
  clientID: process.env.GOOGLE_CLIENT_ID,
  clientSecret: process.env.GOOGLE_CLIENT_SECRET,
  callbackURL: process.env.CALLBACK_URL
}, async (accessToken, refreshToken, profile, done) => {
  let user = await User.findOne({ googleId: profile.id });

  if (!user) {
    user = await User.create({
      googleId: profile.id,
      displayName: profile.displayName,
      email: profile.emails[0].value,
      photo: profile.photos[0].value
    });
  }

  const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, {
    expiresIn: "1h" });

  return done(null, { user, token });
}));
```

Cập nhật server.js để gửi JWT khi xác thực:

```
javascript
```

```
app.get("/auth/google/callback",
  passport.authenticate("google", { session: false }),
  (req, res) => {
    res.json({ token: req.user.token, user: req.user.user });
  }
);
```

10. Tổng kết

- ◆ **Passport.js** giúp dễ dàng tích hợp xác thực OAuth vào ứng dụng Node.js.
- ◆ **OAuth Providers** như Google, Facebook, GitHub giúp người dùng đăng nhập mà không cần mật khẩu.
- ◆ **Session vs JWT**: Session tốt cho web apps, JWT phù hợp với API.
- ◆ Hệ thống có thể mở rộng với **refresh token**, **role-based authentication**.

KINH NGHIỆM THỰC TIỄN

1 Kinh nghiệm thực tiễn

Vấn đề	Kinh nghiệm
Lựa chọn OAuth Provider	Chọn Google, Facebook, GitHub tùy theo nhu cầu
Lưu Access Token an toàn	Dùng HTTP-only cookie hoặc JWT
Tối ưu User Schema	Lưu <code>provider</code> , <code>providerId</code> , tránh trùng email
Bảo mật Redirect URL	Không dùng <code>localhost</code> trên production, kiểm tra <code>state</code>
Giới hạn quyền (scope)	Chỉ lấy <code>email</code> , <code>profile</code> , không yêu cầu dư thừa
Giới hạn tốc độ API	Sử dụng <code>express-rate-limit</code> hoặc Redis
Xử lý Logout đúng cách	Xóa session (session-based) hoặc xóa token (JWT-based)

1.1. Lựa chọn OAuth Provider phù hợp

- **Google OAuth:** Phổ biến nhất, dễ triển khai.
- **Facebook OAuth:** Hữu ích nếu ứng dụng có yếu tố mạng xã hội.
- **GitHub OAuth:** Phù hợp với các ứng dụng dành cho lập trình viên.
- **Apple OAuth:** Bắt buộc với một số ứng dụng trên iOS.
- **Microsoft OAuth:** Dành cho các ứng dụng liên quan đến hệ sinh thái Microsoft (Teams, Outlook, Azure).

💡 **Kinh nghiệm:** Nếu ứng dụng hỗ trợ nhiều OAuth providers, nên sử dụng **"passport-multi-strategy"** để quản lý.

1.2. Xác thực một lần (SSO - Single Sign-On)

- **Lợi ích:** Người dùng không cần tạo tài khoản mới.
- **Ứng dụng thực tế:** Dùng trong hệ thống nội bộ doanh nghiệp (Google Workspace, Microsoft Azure AD).
- **Cách triển khai:** Kết hợp OAuth với JWT để chia sẻ phiên đăng nhập giữa các dịch vụ.

💡 **Kinh nghiệm:** Nếu ứng dụng có nhiều dịch vụ con, sử dụng **OAuth cùng JWT** để duy trì trạng thái đăng nhập.

1.3. Lưu trữ Access Token & Refresh Token đúng cách

- **Session-based authentication:** Lưu phiên vào **session** trong Express.
- **JWT-based authentication:** Trả về **JWT token** sau khi xác thực.
- **Refresh Token:** Lưu trữ trong database hoặc cookie HTTP-only để làm mới Access Token.

💡 **Kinh nghiệm:**

- **Không lưu Access Token trong LocalStorage** (dễ bị tấn công XSS).
- **Sử dụng HTTP-only cookie** để lưu Refresh Token.
- **Thiết lập token expiration ngắn hạn (15-30 phút)** để giảm rủi ro bị đánh cắp.

1.4. Tối ưu hóa Database khi lưu thông tin User

- Chỉ lưu **các trường cần thiết** như `googleId`, `email`, `displayName`, `photo`.
- **Không lưu Access Token vào database**, trừ khi cần sử dụng API của OAuth Provider.
- Cấu trúc User Schema tốt hơn:

```
javascript

const UserSchema = new mongoose.Schema({
  provider: { type: String, enum: ["google", "facebook", "github"],
    required: true },
  providerId: { type: String, required: true, unique: true },
  displayName: String,
  email: { type: String, unique: true },
  photo: String,
  createdAt: { type: Date, default: Date.now }
});
```

💡 Kinh nghiệm:

- Nếu hỗ trợ **đa OAuth providers**, cần lưu `provider` và `providerId`.
- Dùng **unique index trên email** để tránh tạo trùng user.

1.5. Kết hợp OAuth với Role-based Authentication (RBAC)

- Sau khi xác thực bằng OAuth, **cần gán role** cho user (`admin`, `user`, `editor`...).
- Có thể lưu role vào database hoặc kiểm soát qua JWT payload.

💡 Kinh nghiệm:

- Nếu user mới đăng nhập lần đầu, cần gán **default role** (`user`).
- Sử dụng middleware để kiểm tra quyền truy cập:

```
javascript

const checkRole = (role) => (req, res, next) => {
  if (req.user?.role !== role) return
  res.status(403).send("Forbidden");
  next();
};
```

2 Vấn đề trọng tâm cần lưu ý

2.1. Xử lý lỗi callback từ OAuth Provider

- Nếu người dùng hủy xác thực giữa chừng, cần xử lý lỗi khi `/auth/google/callback` bị lỗi.
- Thêm xử lý lỗi trong middleware:

```

javascript

app.get("/auth/google/callback",
  passport.authenticate("google", { failureRedirect: "/" }),
  (req, res) => {
    res.redirect("/dashboard");
  }
);

```

💡 **Kinh nghiệm:** Luôn có **failureRedirect** để tránh lỗi trải nghiệm người dùng.

2.2. Bảo mật OAuth Redirect URL

- OAuth Redirect URL cần được cấu hình chính xác trong Google Developer Console hoặc Facebook Developer.
- **Không nên để redirect URL là localhost trong môi trường production.**
- Luôn kiểm tra **state** để chống CSRF khi xác thực OAuth.

💡 **Kinh nghiệm:**

- Dùng **environment variable** để đặt `CALLBACK_URL`.
- Bật **Strict Mode** trong cài đặt OAuth Provider để tránh redirect URL giả mạo.

2.3. Giới hạn quyền truy cập OAuth Scope

- **Không nên yêu cầu quá nhiều quyền (scope)** trừ khi cần thiết.
- Ví dụ, chỉ nên yêu cầu **profile** và **email**:

```

javascript

passport.authenticate("google", { scope: ["profile", "email"] })

```

💡 **Kinh nghiệm:** Nếu cần truy cập API của Google (YouTube, Drive...), **chỉ lấy refresh token nếu cần.**

2.4. Giới hạn tốc độ API để tránh bị lạm dụng

- Nếu server bị spam bởi OAuth login requests, có thể bị khóa API.
- Dùng **rate-limiting middleware** để bảo vệ API:

```

npm install express-rate-limit
javascript

const rateLimit = require("express-rate-limit");
const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 phút
  max: 100 // Giới hạn 100 request
});
app.use("/auth", authLimiter);

```

💡 **Kinh nghiệm:** Nếu ứng dụng lớn, sử dụng **Redis hoặc Cloudflare Rate Limiting** để kiểm soát.

2.5. Xử lý Logout đúng cách

- Nếu dùng session, cần gọi `req.logout()` và hủy session:

```
javascript

app.get("/logout", (req, res) => {
  req.logout() => {
    req.session.destroy();
    res.redirect("/");
  });
});
```

- Nếu dùng JWT, chỉ cần xóa token ở phía client.

💡 **Kinh nghiệm:** Nếu dùng OAuth + JWT, **không thể logout trên OAuth Provider** vì token không bị xóa.

3 💡 Gợi ý mở rộng:

- Tích hợp **JWT Refresh Token** để không cần đăng nhập lại sau khi Access Token hết hạn.
- Dùng **RBAC (Role-Based Access Control)** để phân quyền theo vai trò người dùng.
- Hỗ trợ đa **OAuth providers** (Google, Facebook, GitHub, Apple, Microsoft).

EXERCISE

OAuth Authentication Server

1. Tóm Tắt Thông Tin Dự Án

- **Mục tiêu:** Xây dựng một server Node.js hỗ trợ xác thực người dùng bằng Google OAuth 2.0.
- **Công nghệ sử dụng:** Node.js, Express.js, Passport.js, MongoDB, Mongoose, dotenv, session, ejs.
- **Chức năng chính:**
 - Đăng nhập với Google OAuth.
 - Lưu thông tin người dùng vào MongoDB.
 - Quản lý phiên đăng nhập.
 - Đăng xuất an toàn.
- **Cấu trúc thư mục cho**

```
oauth-server/  
├── node_modules/           # Thư viện cài đặt qua npm  
├── config/                 # Cấu hình hệ thống  
│   ├── db.js              # Kết nối MongoDB  
│   └── passport.js        # Cấu hình Passport OAuth  
├── models/                # Định nghĩa Schema của MongoDB  
│   └── User.js            # Model cho người dùng  
├── routes/                # Các route của ứng dụng  
│   └── auth.js            # Routes liên quan đến xác thực OAuth  
├── views/                 # Tập giao diện EJS  
│   ├── index.ejs         # Trang chính  
│   ├── dashboard.ejs     # Trang Dashboard sau khi đăng nhập  
│   └── login.ejs         # Trang đăng nhập  
├── public/                # Tập tĩnh như CSS, JS, hình ảnh  
├── .env                   # Biến môi trường (Google Client ID, Mongo URI,  
...)  
├── .gitignore             # Danh sách tệp/thư mục bỏ qua khi đẩy lên Git  
├── package.json           # Thông tin dự án và danh sách dependencies  
├── server.js              # File khởi chạy server chính  
└── README.md              # Hướng dẫn sử dụng và thiết lập dự án
```

Giải thích các thư mục chính:

- **config/:** Chứa các file cấu hình như kết nối MongoDB (`db.js`) và thiết lập OAuth với Passport (`passport.js`).
- **models/:** Chứa model dữ liệu của người dùng (`User.js`).
- **routes/:** Chứa các route xử lý xác thực OAuth (`auth.js`).
- **views/:** Chứa các file giao diện sử dụng EJS để render HTML.
- **public/:** Chứa tài nguyên tĩnh như CSS, JS.
- **.env:** Lưu trữ các thông tin bảo mật như Google OAuth Client ID, MongoDB URI.
- **server.js:** File chính để khởi chạy server Express.
-

2. Các Bước Triển Khai

1. Cài đặt môi trường phát triển

2. **Tạo project Node.js và cài đặt các thư viện cần thiết**
3. **Thiết lập Express server**
4. **Kết nối MongoDB**
5. **Cấu hình Passport.js với Google OAuth**
6. **Tạo routes cho xác thực OAuth**
7. **Quản lý phiên đăng nhập với session**
8. **Tạo giao diện hiển thị bằng EJS**
9. **Kiểm tra và đánh giá kết quả**
10. **Xử lý lỗi ngoại lệ thường gặp**

3. Chi Tiết Từng Bước Thực Hiện

Bước 1: Cài Đặt Môi Trường Phát Triển

- Cài đặt **Node.js** (nếu chưa có): <https://nodejs.org/en/download/>
- Cài đặt **MongoDB** (nếu chưa có): <https://www.mongodb.com/try/download/community>

Bước 2: Tạo Project Node.js và Cài Đặt Các Thư Viện Cần Thiết

```
mkdir oauth-server
cd oauth-server
npm init -y
npm install express mongoose passport passport-google-oauth20 express-session
dotenv ejs
```

Bước 3: Thiết Lập Express Server

Tạo file `server.js`:

```
const express = require('express');
const mongoose = require('mongoose');
const session = require('express-session');
const passport = require('passport');
require('dotenv').config();
require('./config/passport'); // Cấu hình Passport

const app = express();

// Middleware
app.use(express.urlencoded({ extended: true }));
app.use(session({ secret: 'secret', resave: false, saveUninitialized: false }));
app.use(passport.initialize());
app.use(passport.session());

// Routes
app.use(require('./routes/auth')); // Route xác thực OAuth

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server chạy trên cổng ${PORT}`));
```

Bước 4: Kết Nối MongoDB

Tạo file `.env` để lưu biến môi trường:

```
MONGO_URI=mongodb://localhost:27017/oauth_demo
GOOGLE_CLIENT_ID=your_google_client_id
GOOGLE_CLIENT_SECRET=your_google_client_secret
CALLBACK_URL=http://localhost:5000/auth/google/callback
```

Tạo file config/db.js để kết nối MongoDB:

```
const mongoose = require('mongoose');
const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log('MongoDB Connected');
  } catch (error) {
    console.error(error);
  }
};
module.exports = connectDB;
```

Thêm vào server.js:

```
const connectDB = require('./config/db');
connectDB();
```

Bước 5: Cấu Hình Passport.js với Google OAuth

Tạo file config/passport.js:

```
const passport = require('passport');
const GoogleStrategy = require('passport-google-oauth20').Strategy;
const User = require('../models/User');

passport.use(new GoogleStrategy({
  clientID: process.env.GOOGLE_CLIENT_ID,
  clientSecret: process.env.GOOGLE_CLIENT_SECRET,
  callbackURL: process.env.CALLBACK_URL
}, async (accessToken, refreshToken, profile, done) => {
  let user = await User.findOne({ googleId: profile.id });
  if (!user) {
    user = await User.create({ googleId: profile.id, displayName:
profile.displayName });
  }
  done(null, user);
}));

passport.serializeUser((user, done) => {
  done(null, user.id);
});

passport.deserializeUser(async (id, done) => {
  const user = await User.findById(id);
  done(null, user);
});
```

Bước 6: Tạo Routes Cho Xác Thực OAuth

Tạo file routes/auth.js:

```
const express = require('express');
const passport = require('passport');
const router = express.Router();
```

```

router.get('/auth/google', passport.authenticate('google', { scope:
['profile'] }));
router.get('/auth/google/callback', passport.authenticate('google', {
failureRedirect: '/' })), (req, res) => {
    res.redirect('/dashboard');
});
router.get('/logout', (req, res) => {
    req.logout(() => res.redirect('/'));
});

module.exports = router;

```

Bước 7: Quản Lý Phiên Đăng Nhập Với Session

- `express-session` giúp lưu trạng thái đăng nhập.
- Passport sẽ sử dụng session để giữ thông tin người dùng.

Bước 8: Tạo Giao Diện Hiển Thị Bằng EJS

Cài đặt EJS:

```
npm install ejs
```

Tạo thư mục `views/` và file `views/index.ejs`:

```

<!DOCTYPE html>
<html>
<head>
    <title>OAuth Demo</title>
</head>
<body>
    <h1>Đăng nhập với Google</h1>
    <a href="/auth/google">Login with Google</a>
</body>
</html>

```

Bước 9: Kiểm Tra Và Đánh Giá Kết Quả

Chạy server:

```
node server.js
```

Truy cập `http://localhost:5000` để kiểm tra.

Bước 10: Xử Lý Lỗi Ngoại Lệ Thường Gặp

1. **Lỗi Redirect URI mismatch** → Kiểm tra `CALLBACK_URL` trong Google Developer Console.
2. **Lỗi Unauthorized Client** → Đảm bảo ứng dụng đã bật OAuth consent screen.
3. **Lỗi session bị mất sau khi đăng nhập** → Kiểm tra cài đặt `express-session`.